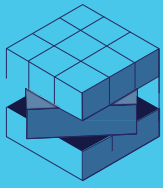


Proyecto 1 – Entrega 4

Contenido

Proyecto 1 – Entrega 4.....	1
Integrantes	2
Entregables	2
Arquitectura del ambiente	3
Configuración de New Relic.....	3
Evidencias y análisis.....	5
Tiempo de respuesta de los servicios	5
Tiempo de respuesta de base de datos	6
Apdex	7
Errores.....	9
Alertas.....	10
Pruebas ejecutadas.....	11
Pruebas funcionales sobre endpoints	11
Prueba de estrés sobre ambiente	11
Tiempo de respuesta de los servicios	13
Apdex	14
Conclusiones	15

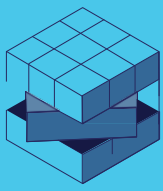


Integrantes

Nombre	Correo
Alejandra Bravo	a.bravo@uniandes.edu.co
Martin Flores	r.floresa@uniandes.edu.co
Carlos Niño	cf.ninor1@uniandes.edu.co
Juan Rodriguez	j.rodriguezg@uniandes.edu.co

Entregables

Entregable	Enlace
Video sustentación	https://youtu.be/VTQRLB7tOg
Documentación Postman	https://documenter.getpostman.com/view/34079512/2sBXqNkxz8
Collection	https://github.com/cfninor/MISW-4304-PipelineCrew/blob/main/PipelineCrew%20-%20Blacklist%20API.postman_collection.json
Endpoint servicios	lb-python-app-1781400546.us-east-2.elb.amazonaws.com
Repositorio GitHub	https://github.com/cfninor/MISW-4304-PipelineCrew



Arquitectura del ambiente

Elemento	Valor
Nombre del microservicio	Blacklist API
Lenguaje / framework	Python / Flask
Plataforma cloud	AWS Fargate
Servicio de contenedores	ECS
Repositorio	CodeCommit
Pipeline CI/CD	CodePipeline + CodeBuild + CodeDeploy
Base de datos	PostgreSQL / RDS
Herramienta de monitoreo	New Relic
Nombre de la app en New Relic	PipelineCrew-Blacklist-Cloud
Endpoints monitoreados	POST /blacklists, GET /blacklists/{email}

Configuración de New Relic

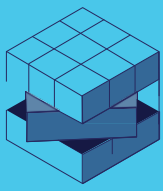
En esta evidencia se observa el archivo `newrelic.ini`, utilizado para configurar el agente de New Relic dentro del microservicio. En este archivo se definió el nombre de la aplicación monitoreada como `PipelineCrew-Blacklist-Cloud`, se activó el modo de monitoreo, el trazado de transacciones y la recolección de errores.

≡ `newrelic.ini` ✕

≡ `newrelic.ini`

```
1  [newrelic]
2  license_key = 90898b9b57d6552cfc0b26b7449ea2dc5893NRAL
3  host = collector.newrelic.com
4  app_name = PipelineCrew-Blacklist-Cloud
5  monitor_mode = true
6  log_file = stderr
7  log_level = info
8  ssl = true
9  high_security = false
10 transaction_tracer.enabled = true
11 transaction_tracer.transaction_threshold = apdex_f
12 transaction_tracer.record_sql = obfuscated
13 transaction_tracer.threshold = 2.0
14 error_collector.enabled = true
15 error_collector.ignore_errors =
16 browser_monitoring.auto_instrument = true
17 thread_profiler.enabled = true
```

En el `dockerfile` se evidencia que el contenedor copia el código fuente de la aplicación, instala las dependencias del proyecto y declara la variable `NEW_RELIC_CONFIG_FILE=/app/newrelic.ini`. Esta variable indica al agente de New Relic la ubicación del archivo de configuración que será utilizado al iniciar la aplicación.



Dockerfile X

Dockerfile > FROM

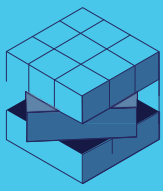
```
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN python -m pip install --upgrade pip setuptools wheel \
8     && python -m pip install --ignore-installed -r requirements.txt
9
10 COPY . .
11
12 EXPOSE 5000
13
14 ENV NEW_RELIC_CONFIG_FILE=/app/newrelic.ini
15
16 CMD ["python3", "application.py"]
```

En el archivo de application.py se inicializa el agente de New Relic antes de crear la aplicación Flask. Para eso se importa newrelic.agent y se ejecuta newrelic.agent.initialize('newrelic.ini'). De esta forma, cuando la aplicación inicia el agente va a quedar cargado y puede reportar transacciones, errores y métricas hacia New Relic.

application.py 1 X

application.py > ...

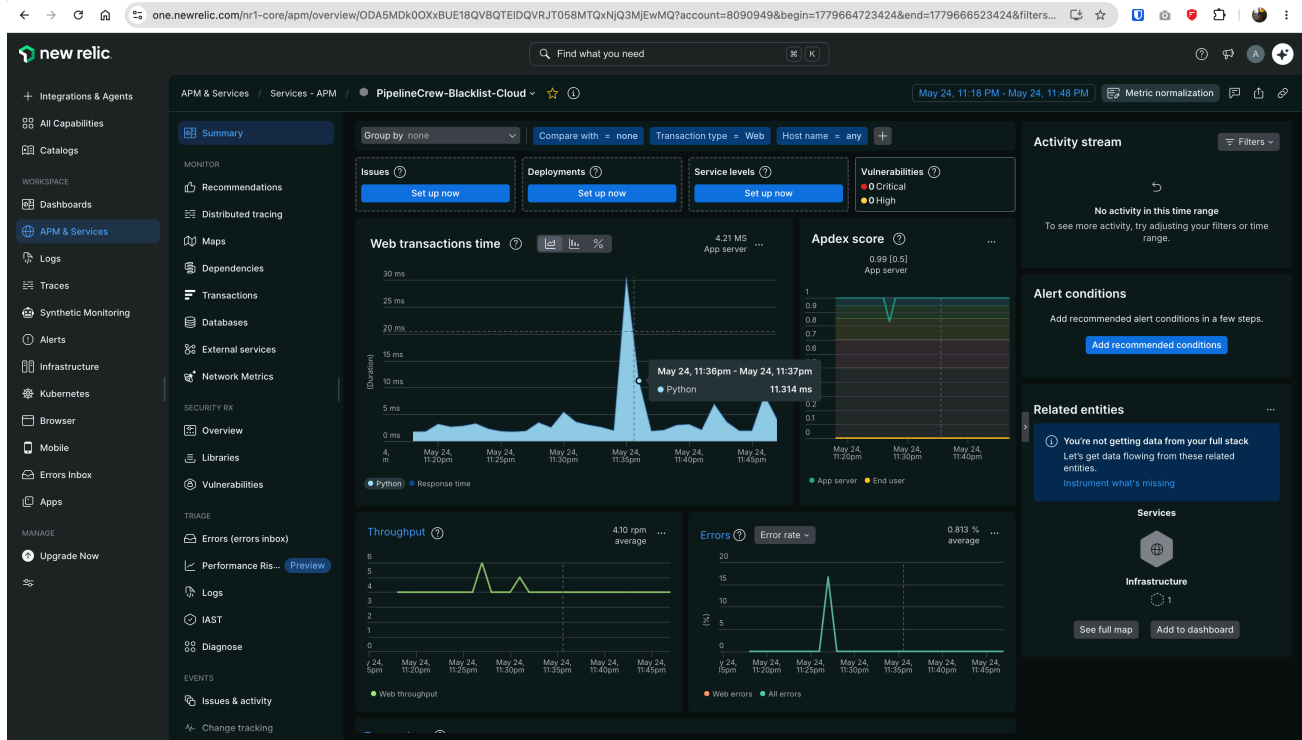
```
1 import newrelic.agent
2 try:
3     newrelic.agent.initialize('newrelic.ini')
4 except Exception:
5     pass
6
7 from app import create_app
8
9 application = create_app()
10 #print(application.url_map)
11
12 if __name__ == "__main__":
13     application.run(host="0.0.0.0", port=5000, debug=True)
```



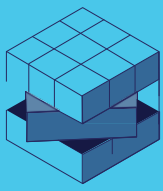
Evidencias y análisis

Tiempo de respuesta de los servicios

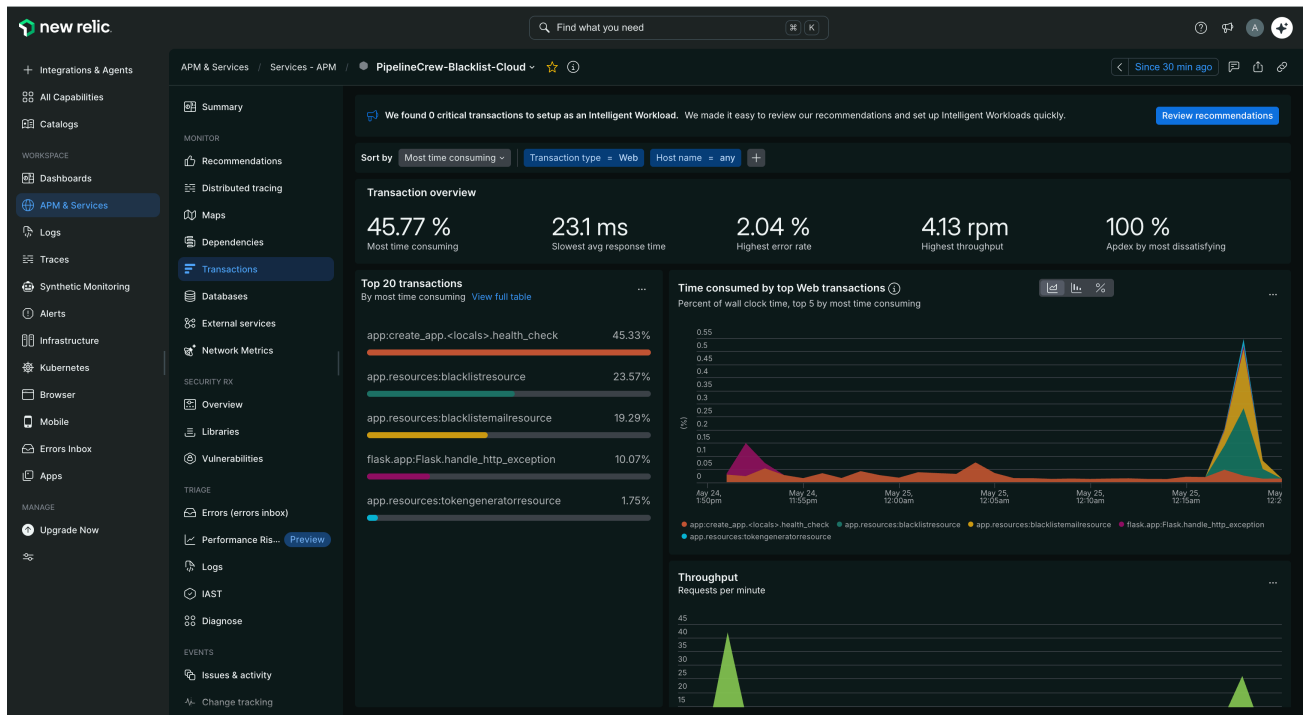
Después de ejecutar peticiones funcionales sobre los endpoints del microservicio, New Relic registró actividad en la aplicación. En la evidencia se observan métricas de tráfico reciente, tiempo de respuesta y throughput, lo que permite confirmar que la instrumentación está enviando eventos hacia New Relic APM.



En la sección de transacciones se observan los endpoints ejecutados durante las pruebas funcionales. Esta vista permite identificar qué operaciones del microservicio consumen más tiempo, cuáles reciben mayor cantidad de peticiones y cuáles pueden afectar el desempeño general de la aplicación.



DevOps: Agilizando el despliegue continuo de aplicaciones

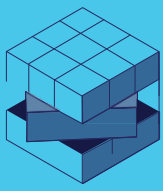


En la vista de transacciones de New Relic se observan las operaciones web capturadas por el agente APM. Aunque New Relic no muestra los endpoints con la ruta exacta de la URL, si registra las transacciones asociadas a los recursos internos de la aplicación, como blacklistresource y blacklistemailresource, que corresponden a las operaciones principales del microservicio de listas negras.

Transaction	Total ti...	Avg	Min	Max	Median...	95th...	99th...	Ap...	Error r...	Throug...	Total c...	Key transaction
app.resources:bl...	22.10%	18.9 ms	0 ms	49.4 ms	3.39 ms	49.4 ms	49.4 ms	1	0%	0.3	10	NA
app.create_app...	47.64%	3.24 ms	1.47 ms	47.6 ms	1.8 ms	9.64 ms	24.7 ms	1	0%	4.20	126	NA
flask.app.Flask.h...	1.26%	2.15 ms	0 ms	2.55 ms	2.01 ms	2.55 ms	2.55 ms	1	0%	0.2	5.00	NA
app.resources:bl...	27.00%	23.1 ms	0 ms	63.9 ms	5.53 ms	63.9 ms	63.9 ms	1	0%	0.3	10	NA
app.resources:to...	2.00%	3.43 ms	0 ms	7.93 ms	2.24 ms	7.93 ms	7.93 ms	1	0%	0.2	5.00	NA

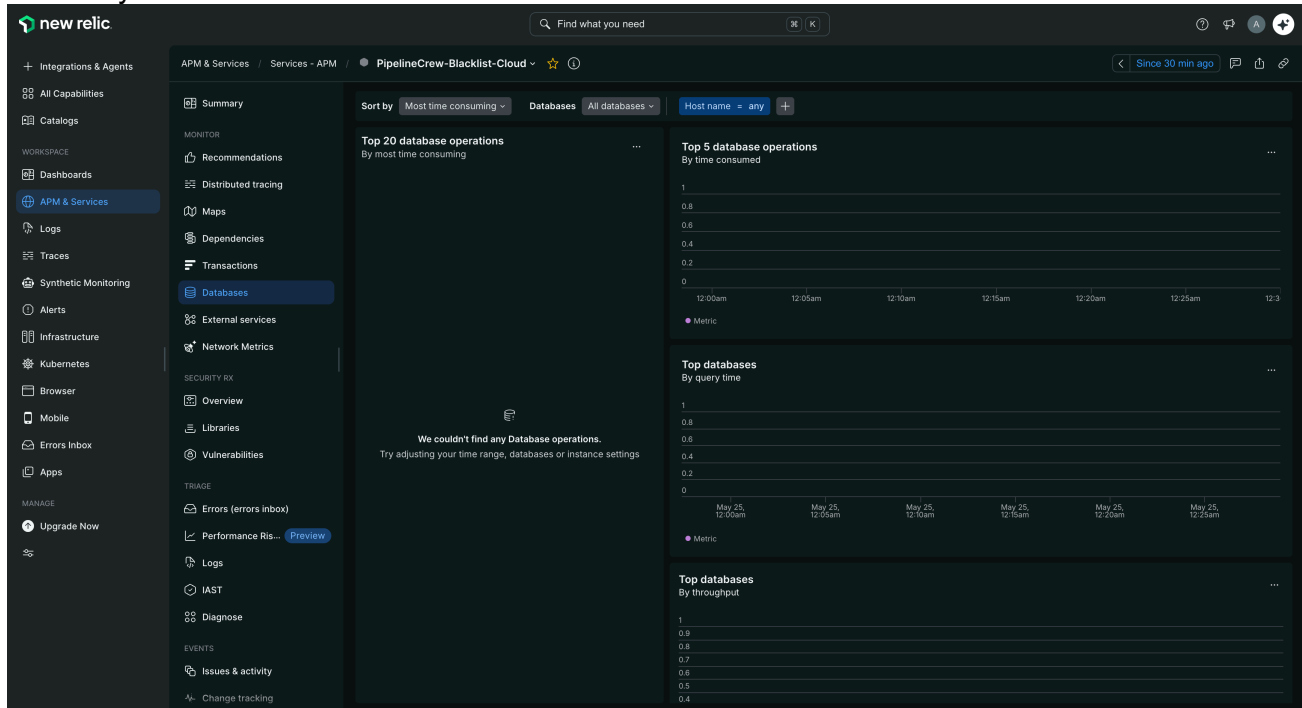
Tiempo de respuesta de base de datos

En la vista de Summary de New Relic se observa que el tiempo de respuesta de las transacciones web se descompone por componentes de ejecución, incluyendo Python y Postgres.



DevOps: Agilizando el despliegue continuo de aplicaciones

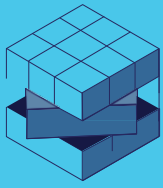
Durante el periodo analizado, New Relic permitió identificar la participación de Postgres dentro del tiempo total de respuesta de la aplicación. Aunque la vista específica de Databases no mostró operaciones desagregadas por consulta, la vista Summary sí permitió identificar el componente Postgres dentro del tiempo de transacción. Por lo tanto, el análisis de DB se soporta con la descomposición del tiempo de respuesta observada en Summary.



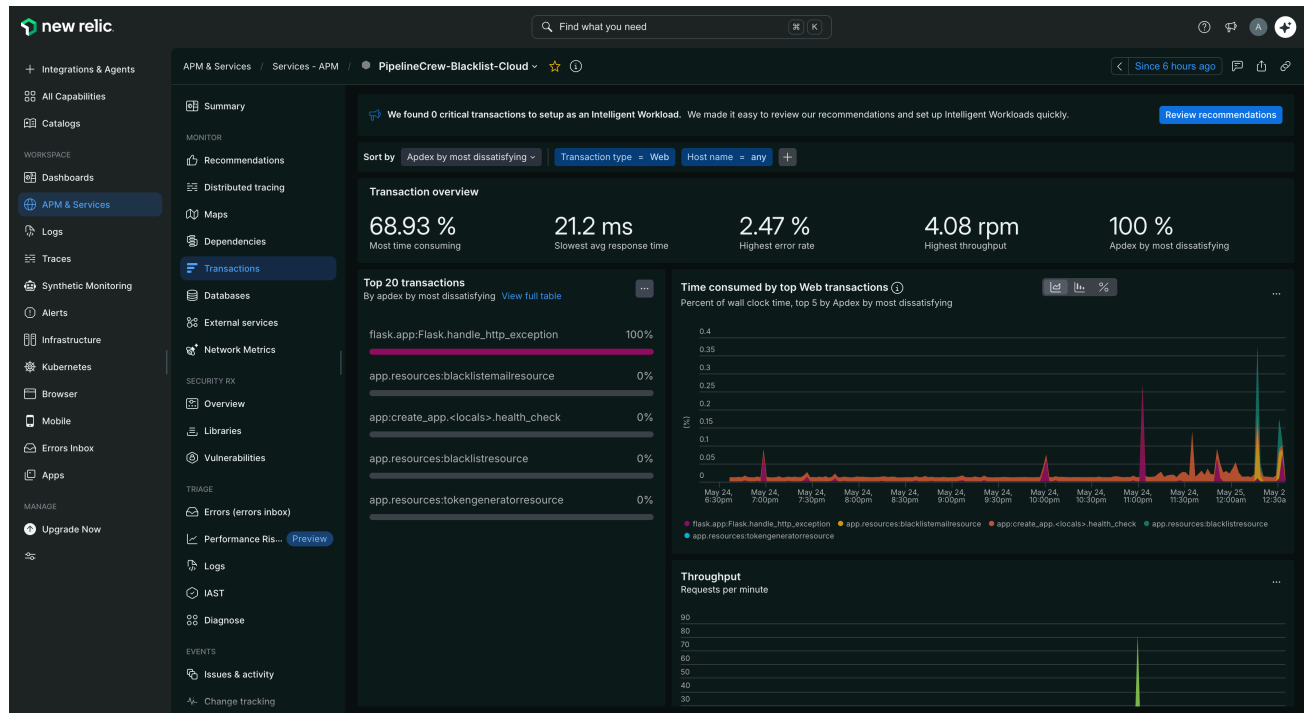
Aunque el microservicio utiliza persistencia, New Relic no presentó métricas desagregadas de base de datos en esta vista, lo que indica que, con la configuración actual del agente, la herramienta está capturando transacciones web, throughput, Apdex, errores, pero no está mostrando el detalle de las consultas o tiempos específicos de DB. Por lo tanto, el impacto de la base de datos solo puede observarse indirectamente dentro del tiempo total de la respuesta de las transacciones.

Apdex

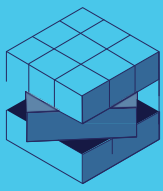
En esta evidencia se observa la vista de transacciones ordenada por Apdex by most dissatisfying. New Relic permite identificar qué transacciones afectan en mayor medida la satisfacción del usuario según el tiempo de respuesta esperado. Durante el periodo analizado, la transacción con mayor impacto negativo fue flask.app:Flask.handle_http_exception, asociada a errores HTTP generados por peticiones inválidas o no exitosas.



DevOps: Agilizando el despliegue continuo de aplicaciones



Durante las pruebas también se ejecutaron escenarios no exitosos de la colección de Postman, lo que permitió observar cómo las excepciones HTTP pueden impactar la vista de Apdex y la satisfacción de las transacciones monitoreadas.



DevOps: Agilizando el despliegue continuo de aplicaciones

PipelineCrew - Blacklist API - Run results

Ran today at 07:32:59 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	1s 363ms	35	0	174 ms

All 35 Passed 31 Failed 4 Skipped 0 Errors 0 Console log

[List](#) [Grid](#)

<http://lb-python-app-1781400546.us-east-2.elb.amazonaws.com/blacklists/> 404 • 114 ms • 394 B 6

- PASS Existe token para autenticación
- PASS Status code dentro de los esperados para email no blacklistado
- PASS Tiempo de respuesta menor a 5000 ms
- PASS Existe token en variable de colección para request autenticado
- PASS La respuesta es coherente con un email válido no bloqueado
- PASS Si el body expone el email, corresponde al consultado

GET 5. Verificar email con formato inválido

<http://lb-python-app-1781400546.us-east-2.elb.amazonaws.com/blacklists/invalidemail> 400 • 115 ms • 235 B 6

- PASS Existe token para autenticación
- PASS Status code dentro de los esperados para email inválido
- PASS Tiempo de respuesta menor a 5000 ms
- PASS Existe token en variable de colección para request autenticado
- PASS La respuesta comunica error o validación de formato inválido
- PASS Si el body expone el valor inválido, corresponde al consultado

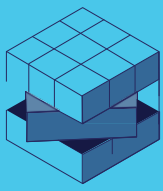
POST 6. Agregar email a blacklist (sin app_uuid)

<http://lb-python-app-1781400546.us-east-2.elb.amazonaws.com/blacklists> 400 • 210 ms • 221 B 4

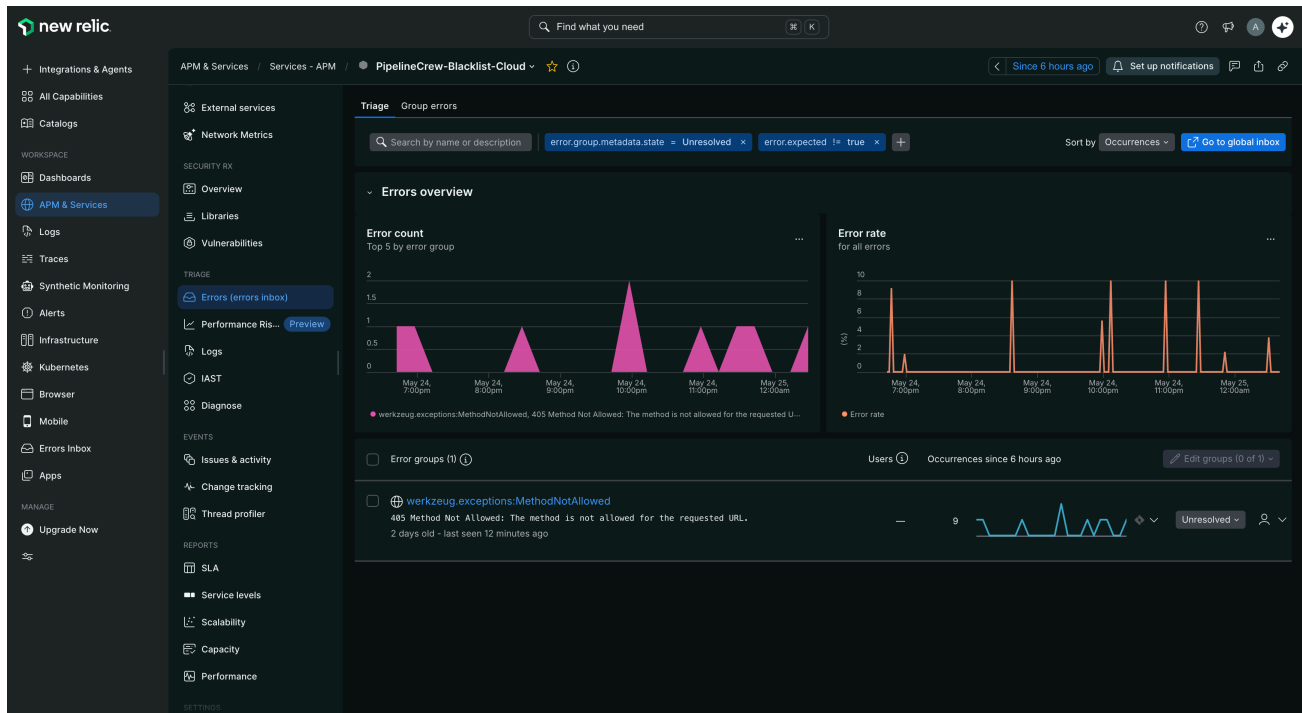
- PASS Existe token para autenticación
- PASS Status code 401 esperado para crear/agregar email

Errores

En la vista Errors Inbox de New Relic se observa el registro de errores generados por la aplicación PipelineCrew-Blacklist-Cloud. La herramienta agrupó los errores bajo el tipo `werkzeug.exceptions.MethodNotAllowed`, correspondiente a respuestas HTTP 405 Method Not Allowed. Este error se presenta cuando se realiza una petición a un endpoint utilizando un método HTTP no permitido para esa URL.

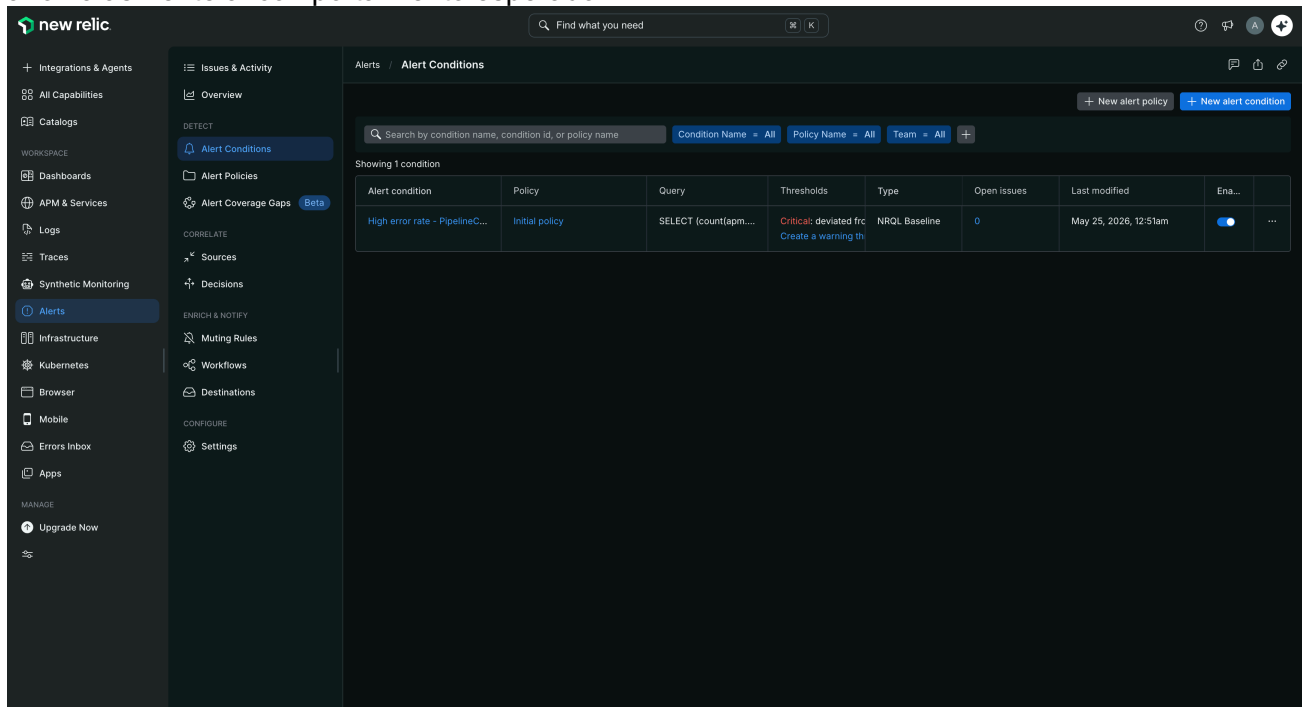


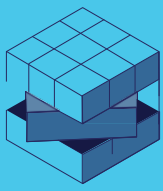
DevOps: Agilizando el despliegue continuo de aplicaciones



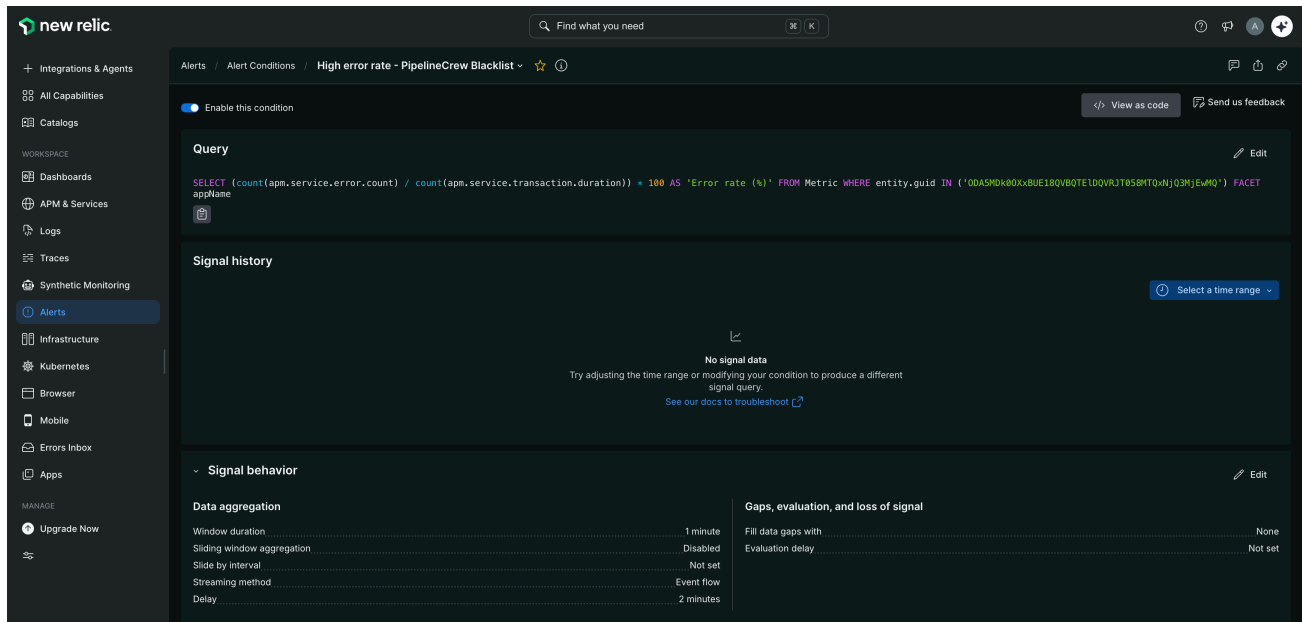
Alertas

En esta evidencia se observa la configuración de una condición de alerta de New Relic para la aplicación monitoreada. La condición fue creada bajo el nombre High error rate - PipelineCrew Blacklist, asociada a la política Initial Policy, con tipo NRQL Baseline y estado activo. Esta alerta permite monitorear el comportamiento de errores de la aplicación y detectar desviaciones anómalas frente al comportamiento esperado.





DevOps: Agilizando el despliegue continuo de aplicaciones



New Relic permite configurar alertas personalizadas a partir de consultas NRQL. En este caso, la alerta quedó habilitada para monitorear el error rate de la aplicación. Al momento de la captura, New Relic mostró No signal Data, por lo que la condición estaba configurada, pero no se había disparado para evaluación en ese rango.

Pruebas ejecutadas

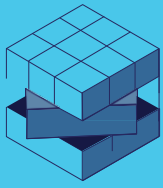
Pruebas funcionales sobre endpoints

Se ejecutaron pruebas funcionales sobre los endpoints principales del microservicio con el fin de generar tráfico real hacia la aplicación desplegada. Estas pruebas permitieron validar los escenarios exitosos y no exitosos de la API, además de generar los eventos observables en New Relic.

Endpoint	Método	Escenario	Resultado esperado
/blacklists	POST	Registrar email válido	Email agregado correctamente
/blacklists/{email}	GET	Consultar email registrado	Retorna estado del email
/blacklists/{email}	GET	Consultar email no registrado	Retorna que no está en blacklist
/blacklists	POST / GET incorrecto	Generar error controlado	Error HTTP 405 o error esperado

Prueba de estrés sobre ambiente

La prueba de estrés se ejecutó sobre el ambiente cloud desplegado en AWS Fargate, usando el endpoint público del balanceador de carga. El objetivo fue generar tráfico suficiente sobre los servicios principales para observar en New Relic el comportamiento de tiempos de respuesta, throughput, Apdex, errores y operaciones de base datos.



DevOps: Agilizando el despliegue continuo de aplicaciones



Parámetro	Valor
Ambiente probado	AWS Fargate
URL base	lb-python-app-1781400546.us-east-2.elb.amazonaws.com
Herramienta usada	Postman Runner / Newman
Endpoints probados	/blacklists, /blacklists/{email}
Número de iteraciones	300
Delay entre peticiones	0 ms

PipelineCrew - Blacklist API - Run results

Ran today at 08:09:14 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	300	6m 4s	10500	0	190 ms

All 10500 Passed 9299 Failed 1201 Skipped 0 Errors 0 Console log

ListGrid

PASS Si el body expone el email, corresponde al esperado

266267268

GET 3. Verificar si email está en blacklist (GET)

http://lb-python-app-1781400546.us-east-2.elb.amazonaws.com/blacklists/blacklist_1779671718166_36278@example.com

500 • 142 ms • 234 B 6 2

269270271272273274275276277278279280281

PASS Existe token para autenticación

FAIL Status code es 200 | AssertionError: expected 500 to equal 200

PASS Tiempo de respuesta menor a 5000 ms

PASS Existe token en variable de colección para request autenticado

PASS La respuesta tiene contenido útil

PASS Si responde JSON, el body es JSON válido

FAIL La respuesta sugiere que el email consultado está en blacklist | AssertionError: expected false to be true

PASS Si el body expone el email, corresponde al consultado

GET 4. Verificar email válido (no en blacklist)

http://lb-python-app-1781400546.us-east-2.elb.amazonaws.com/blacklists/

404 • 133 ms • 394 B 6

282283284285286287288289290291

PASS Existe token para autenticación

PASS Status code dentro de los esperados para email no blacklisted

PASS Tiempo de respuesta menor a 5000 ms

PASS Existe token en variable de colección para request autenticado

PASS La respuesta es coherente con un email válido no bloqueado

PASS Si el body expone el email, corresponde al consultado

GET 5. Verificar email con formato inválido

http://lb-python-app-1781400546.us-east-2.elb.amazonaws.com/blacklists/invalidemail

400 • 145 ms • 235 B 6

292293294295296297298299300

PASS Existe token para autenticación

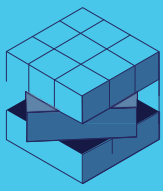
PASS Status code dentro de los esperados para email inválido

PASS Tiempo de respuesta menor a 5000 ms

PASS Existe token en variable de colección para request autenticado

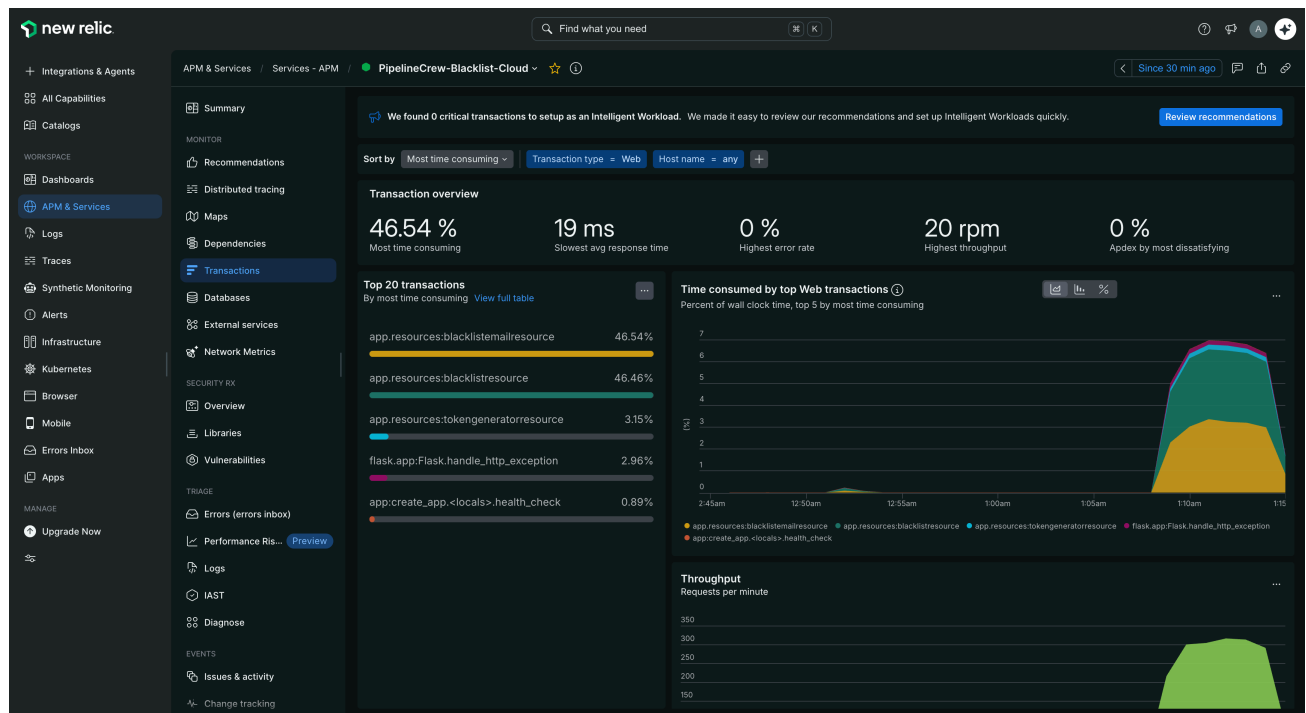
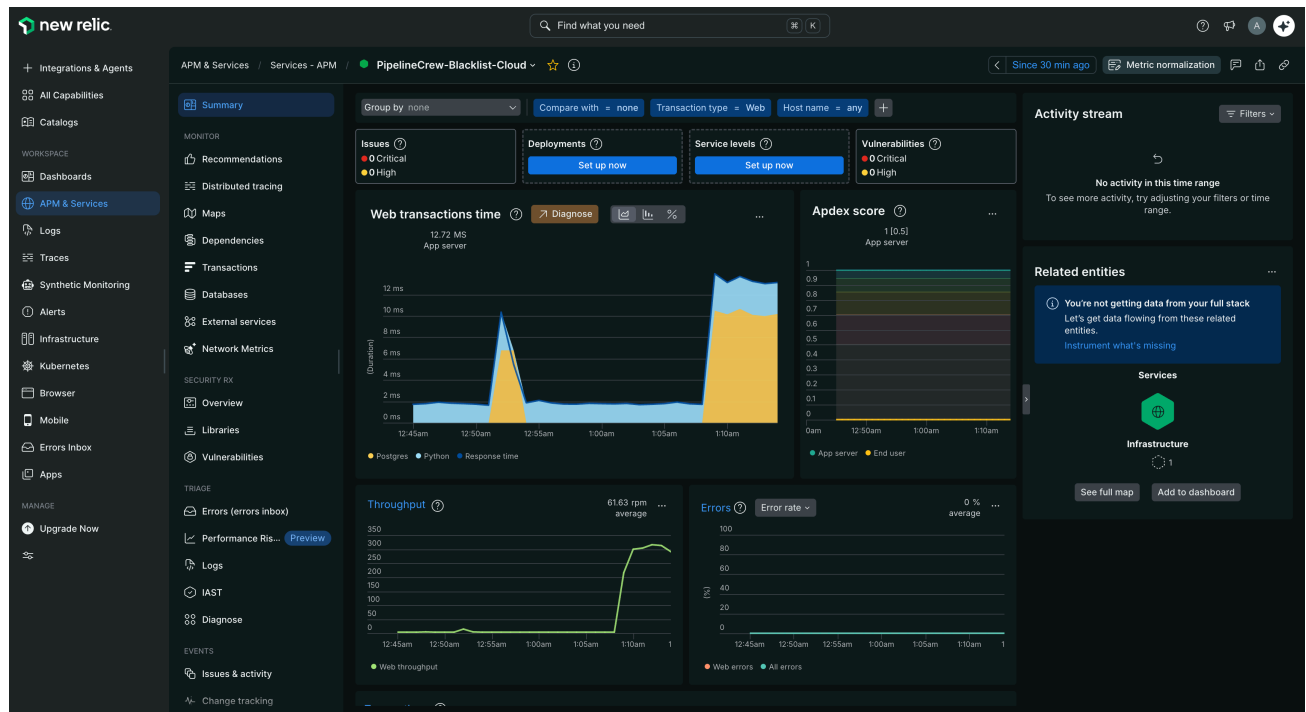
PASS La respuesta comunica error o validación de formato inválido

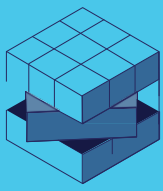
PASS Si el body expone el valor inválido, corresponde al consultado



DevOps: Agilizando el despliegue continuo de aplicaciones

Tiempo de respuesta de los servicios





DevOps: Agilizando el despliegue continuo de aplicaciones

The screenshot shows the New Relic APM & Services interface for the service 'PipelineCrew-Blacklist-Cloud'. The left sidebar contains navigation options like Integrations & Agents, All Capabilities, Catalogs, Workspace, Dashboards, APM & Services (selected), Logs, Traces, Synthetic Monitoring, Alerts, Infrastructure, Kubernetes, Browser, Mobile, Errors Inbox, Apps, Upgrade Now, and Change tracking. The main panel displays a 'Summary' view with a message: 'We found 0 critical transactions to setup as an Intelligent Workload. We made it easy to review our recommendations and set up Intelligent Workloads quickly.' Below this, there's a filter for 'Transaction type = Web' and 'Host name = any'. A table titled 'Back to transactions' shows transaction data:

Transaction	Total t...	Avg	Min	Max	Median...	95th ...	99th ...	Ap...	Error r...	Throug...	Total c...	Key transaction
app.resources:bl...	46.55%	19 ms	0 ms	217 ms	30.8 ms	37.6 ms	56.2 ms	1	0%	20	604	NA
app:create_app...	0.88%	1.8 ms	1.44 ms	3.34 ms	1.71 ms	2.29 ms	2.97 ms	1	0%	4.03	121	NA
flask.app:Flask.h...	2.96%	2.42 ms	0 ms	57.9 ms	1.98 ms	3.14 ms	7.93 ms	1	0%	10	302	NA
app.resources:bl...	46.45%	19 ms	0 ms	102 ms	31.3 ms	39.6 ms	60.1 ms	1	0%	20	604	NA
app.resources:to...	3.15%	2.57 ms	0 ms	26.9 ms	2.17 ms	2.91 ms	14.1 ms	1	0%	10	302	NA

Apdex

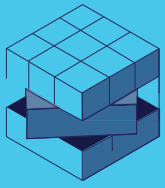
The screenshot shows the New Relic APM & Services interface for the service 'PipelineCrew-Blacklist-Cloud', specifically the 'Apdex' view. The left sidebar is the same as the previous screenshot. The main panel displays 'Transaction overview' with the following metrics:

- 46.55 % Most time consuming
- 19 ms Slowest avg response time
- 0 % Highest error rate
- 20 rpm Highest throughput
- 0 % Apdex by most dissatisfying

Below these metrics, there's a section for 'Top 20 transactions' sorted by 'Apdex by most dissatisfying'. The table shows the following transactions and their Apdex scores:

Transaction	Apdex
app.resources:blacklistemailresource	0%
app:create_app.<locals>.health_check	0%
flask.app:Flask.handle_http_exception	0%
app.resources:blacklistresource	0%
app.resources:tokengeneratorresource	0%

On the right, there's a 'Time consumed by top Web transactions' chart showing the percent of wall clock time for the top 5 transactions by Apdex. The chart shows a significant peak in time consumption around 11:00am. Below this, there's a 'Throughput' chart showing requests per minute, which also peaks around 11:00am.



Conclusiones

- New Relic permitió validar el monitoreo del microservicio desplegado en AWS Fargate, registrando transacciones, tiempos de respuesta, throughput, Apdex, errores y comportamiento asociado a Postgres.
- Las pruebas funcionales y la prueba de estrés permitieron generar tráfico real sobre los endpoints principales del microservicio, lo que facilitó observar el comportamiento de la aplicación desde New Relic APM.
- En el análisis de base de datos, New Relic permitió identificar la participación de Postgres dentro del tiempo total de respuesta, aunque la vista específica de Databases no mostró consultas desagregadas por sentencia SQL.
- La vista de errores permitió identificar excepciones HTTP 405 generadas por escenarios no exitosos, evidenciando la capacidad de New Relic para agrupar y monitorear fallos de aplicación.
- La alerta configurada sobre error rate demuestra que New Relic permite pasar de un monitoreo reactivo a uno proactivo, aunque en la evidencia tomada la condición no se disparó por falta de señal activa en ese rango.